

Response to Updated Review C

Thank you for the careful follow-up. We revised the paper to address both issues: the scope of the undecidability theorem and the unmeasured performance of IntConv detection.

1. Scope and significance of the undecidability result

We agree that the previous version did not state an important limitation clearly enough. The theorem does **not** establish that fixpoint detection is undecidable for every incrementalization of a total, decidable query. It concerns the full DBSP circuit language, which admits unbounded computations that need not converge on every input.

Precisely, if DBSP primitive nodes can express primitive recursive functions, there exists a fixed level-1 circuit c for which no computable detector is both sound and complete on every canonical input $\delta_0(x)$. The proof constructs c from a primitive recursive bounded evaluator. On input encoding a program e , its output stream is identically zero exactly when e does not halt. A sound and complete detector would therefore recognize the complement of the halting problem, which is not recursively enumerable. This result is now formalized in Lean 4.

The construction differs from the setting in your recomputation argument: it does not represent a total query with an independently computable final answer on every input. Its stream-elimination result is defined for nonhalting program codes but undefined for halting codes. Our theorem therefore does not rule out detectors that use a terminating non-incremental algorithm or other query-specific structure.

Moreover, knowing that an incremental computation equals the reference answer whenever it reaches a true fixpoint gives only one implication. Stopping when equality is observed additionally requires the converse: equality with the reference answer must imply that a true fixpoint has been reached. That need not hold for arbitrary DBSP circuits, since an intermediate result may transiently equal the final answer and later change again. When a particular query and incremental algorithm provide the converse, comparison with a non-incremental computation can indeed yield an exact detector.

We retain the theorem because it limits what can be expected for the full circuit language, while emphasizing that useful fragments can behave better. In particular, our later results show that IntConv is sound for all circuits and complete for regular circuits, including Datalog queries, and their incrementally optimized forms. Section 4.2 now states the theorem’s assumptions and its total-query caveat explicitly, and the introduction, overview, and conclusion consistently restrict the undecidability claim to arbitrary expressive circuits.

2. Role and evaluation of IntConv and the StFP detector

We agree that if convergence detection made incremental execution slower than non-incremental recomputation on realistic workloads, it would undermine its practical value. We have not implemented or benchmarked our global IntConv detector, so the paper makes no empirical efficiency claim. The detector may be fast, but our present results do not answer that question.

The contribution of IntConv here is primarily theoretical. It gives a declarative account of the internal-state-stability strategy used in practice. We prove that IntConv soundly implies external convergence, give a sound and complete structural detector for StFP, and use it to establish that

IntConv is decidable. Our completeness results then identify broad circuit classes on which this implementable criterion coincides with the external mathematical specification. These are correctness and expressiveness guarantees rather than performance measurements.

The general StFP detector is therefore a correctness-oriented fallback, not a proposed optimal implementation. Implementations may optimize it or use specialized detectors justified by circuit structure. For example, FirstZero is already sound and complete for the naive and semi-naive query bodies.

Feldera already implements operator-level internal-state-stability checks for its supported programs. What remains unimplemented from our work is the general compositional detector across all bracketed sub-circuits, especially nested ones. The Feldera compiler version we inspected rejects nested recursive operators within recursive code, so evaluating this additional capability would require extending compiler and runtime support rather than merely enabling an existing check.

A meaningful performance evaluation also requires integration into a complete incremental-query system. Detector overhead and comparison with non-incremental recomputation depend on concrete state representations, data structures, scheduling, queries, workloads, update sizes, and other system optimizations abstracted away by our core calculus. An isolated implementation of the structural algorithm would show executability but would not answer the end-to-end performance question.

The revision now makes this scope explicit: Sections 2 and 5 describe the implementation argument as qualitative rather than empirical; Section 6 presents StFP as a general fallback and discusses specialized detectors; and the conclusion identifies implementation, detector-overhead measurement, and end-to-end comparison as future work.